```
================================================================================
CHANGELOG — Code-projet-AI.py
Modifications apportées par Arthur (via Claude Code) par rapport au code de base de Lenny
Projet MLSM2154 — Groupe 6
================================================================================



--------------------------------------------------------------------------------
1. NOUVELLES DÉPENDANCES (imports)
--------------------------------------------------------------------------------

Ajoutées :
    from joblib import Parallel, delayed
    from numba import njit
    from scipy.stats import wilcoxon

Ces trois librairies sont conformes aux consignes : elles ne fournissent pas
d'implémentation de DTW ou d'Edit Distance, elles servent uniquement à
accélérer le calcul (numba, joblib) et à effectuer un test statistique
(scipy). La logique algorithmique reste entièrement codée à la main.


--------------------------------------------------------------------------------
2. ACCÉLÉRATION DE dtw_distance — décorateur @njit(cache=True)
--------------------------------------------------------------------------------

VERSION LENNY :
    def dtw_distance(seq1, seq2):
        ...
        d = np.linalg.norm(seq1[i - 1] - seq2[j - 1])
        g[i, j] = min(
            g[i - 1, j    ] + d,
            g[i - 1, j - 1] + 2 * d,
            g[i,     j - 1] + d,
        )

VERSION ARTHUR :
    @njit(cache=True)
    def dtw_distance(seq1, seq2):
        ...
        dx = seq1[i-1, 0] - seq2[j-1, 0]
        dy = seq1[i-1, 1] - seq2[j-1, 1]
        dz = seq1[i-1, 2] - seq2[j-1, 2]
        d = (dx*dx + dy*dy + dz*dz) ** 0.5
        v    = g[i-1, j  ] + d
        diag = g[i-1, j-1] + 2.0 * d
        h    = g[i,   j-1] + d
        g[i, j] = min(v, min(diag, h))
```

```

Ce qui a changé :
  - Ajout du décorateur @njit(cache=True) : Numba compile la fonction en code
    machine natif lors du premier appel (~5-10s), puis les appels suivants
    s'exécutent à vitesse C. Sur des boucles Python pures de cette taille
    (matrice I×J), le gain est typiquement 50x à 200x.
  - La distance euclidienne est calculée inline (dx, dy, dz manuels) plutôt
    qu'avec np.linalg.norm(), car Numba ne supporte pas toutes les fonctions
    NumPy dans le mode njit.
  - La recurrence elle-même (diagonale ×2, normalisation) est INCHANGÉE.


--------------------------------------------------------------------------------
3. ACCÉLÉRATION DE edit_distance — décorateur @njit(cache=True)
--------------------------------------------------------------------------------

VERSION LENNY :
    def edit_distance(seq1, seq2):
        mat = np.zeros((L1 + 1, L2 + 1), dtype=int)
        mat[:, 0] = np.arange(L1 + 1)
        mat[0, :] = np.arange(L2 + 1)

VERSION ARTHUR :
    @njit(cache=True)
    def edit_distance(seq1, seq2):
        mat = np.zeros((L1 + 1, L2 + 1), dtype=np.int64)
        for i in range(L1 + 1):
            mat[i, 0] = i
        for j in range(L2 + 1):
            mat[0, j] = j

Ce qui a changé :
  - Ajout du décorateur @njit(cache=True), même raison que pour DTW.
  - L'initialisation des bords de la matrice passe de slices NumPy
    (mat[:, 0] = np.arange(...)) à des boucles explicites : Numba njit exige
    des opérations élémentaires, pas de slices vectorisées.
  - Le type de la matrice passe de dtype=int à dtype=np.int64 pour la
    compatibilité explicite avec Numba.
  - La recurrence de Levenshtein (deletion, insertion, substitution) est
    INCHANGÉE.


--------------------------------------------------------------------------------
4. PARALLÉLISATION dans knn_predict et crossval_user_independent
--------------------------------------------------------------------------------

VERSION LENNY — knn_predict :
    distances = np.array([distance_fn(test_item, ref) for ref in train_items])
```

```

VERSION ARTHUR — knn_predict :
    distances = np.array([distance_fn(test_item, ref) for ref in train_items])
    # IDENTIQUE — séquentiel ici

VERSION LENNY — crossval_user_independent :
    preds = []
    for seq_idx, ts in enumerate(test_items):
        print(f"    User {u:2d} | sequence {seq_idx+1:3d}/{len(test_items)} …",
              end="\r", flush=True)
        preds.append(knn_predict(ts, train_items, train_labels, distance_fn, k))

VERSION ARTHUR — crossval_user_independent :
    preds = Parallel(n_jobs=-1, prefer="threads")(
        delayed(knn_predict)(ts, train_items, train_labels, distance_fn, k)
        for ts in test_items
    )
    preds = list(preds)

Ce qui a changé :
  - La boucle séquentielle sur les séquences test est remplacée par un appel
    joblib.Parallel. Cela distribue les prédictions sur tous les cœurs CPU
    disponibles (n_jobs=-1), réduisant le temps d'un facteur proche du nombre
    de cœurs.
  - IMPORTANT : la parallélisation est placée UNIQUEMENT à ce niveau
    (boucle externe sur test_items). knn_predict lui-même reste séquentiel,
    évitant ainsi le problème de pools de threads imbriqués.
  - La ligne de progression (print avec \r) a été supprimée car elle n'est pas
    thread-safe avec joblib.

Même modification appliquée dans compute_confusion_matrix (boucle sur
test_items → Parallel).


--------------------------------------------------------------------------------
5. WARM-UP NUMBA dans le main (section 9.0 — NOUVELLE)
--------------------------------------------------------------------------------

VERSION LENNY : absent

VERSION ARTHUR :
    print("Warming up numba JIT compilation …", end=" ", flush=True)
    _d = np.random.randn(10, 3)
    dtw_distance(_d, _d)
    _s = np.zeros(10, dtype=np.int64)
    edit_distance(_s, _s)
    print("done.")

```

```
Ce qui a changé :
  - Numba compile la fonction au premier appel. Sans ce warm-up, la
    compilation se déclencherait au milieu de la cross-validation, faussant
    la mesure de temps du premier fold. Ce bloc l'exécute sur des données
    fictives avant que la vraie évaluation commence.


--------------------------------------------------------------------------------
6. NOUVELLE FONCTION — wilcoxon_test (section 9 — NOUVELLE)
--------------------------------------------------------------------------------

VERSION LENNY : absent

VERSION ARTHUR :
    def wilcoxon_test(folds_a, folds_b, name_a, name_b, domain):
        stat, p = wilcoxon(folds_a, folds_b)
        print(f"\n  Wilcoxon test — {name_a} vs {name_b} | Domain {domain}")
        print(f"  statistic = {stat:.4f} | p-value = {p:.4f}")
        if p < 0.05:
            better = name_a if np.mean(folds_a) > np.mean(folds_b) else name_b
            print(f"  → Significant difference (p<0.05): {better} is better")
        else:
            print(f"  → No significant difference (p={p:.4f})")

Ce qui a changé :
  - Ajout d'une fonction complète pour le test de Wilcoxon signed-rank,
    exigé explicitement par les consignes (section 5) pour le cas
    user-independent.
  - Appelée dans le main après chaque paire de résultats user-independent
    (DTW vs Edit Distance), pour Domain 1 (section 9.6) et Domain 4
    (section 10.4).


--------------------------------------------------------------------------------
7. PIPELINE DOMAIN 4 COMPLET dans le main (sections 10.1 à 10.5 — NOUVELLES)
--------------------------------------------------------------------------------

VERSION LENNY : Domain 4 était chargé (load_domain4) mais jamais évalué.

VERSION ARTHUR : ajout des sections suivantes pour Domain 4 :

  10.1 — Pré-traitement (standardize_gestures, apply_pca, cluster_and_encode)
  10.2 — Évaluation user-independent (Edit Distance + DTW)
  10.3 — Évaluation user-dependent   (Edit Distance + DTW)
  10.4 — Test de Wilcoxon user-independent Domain 4
  10.5 — Matrice de confusion Edit Distance user-independent Domain 4

Les fonctions appelées sont IDENTIQUES à celles utilisées pour Domain 1 ;
```

```
seuls les arguments (data4, labels4, users4, domain=4) changent.


--------------------------------------------------------------------------------
8. CE QUI N'A PAS CHANGÉ
--------------------------------------------------------------------------------

Les éléments suivants sont strictement identiques entre les deux versions :

  - load_domain1()            — chargement et parsing CSV Domain 1
  - load_domain4()            — chargement et parsing texte Domain 4
  - check_completeness()      — vérification de complétude
  - print_dataset_info()      — statistiques descriptives
  - plot_sequence_lengths()   — boîte à moustaches
  - plot_gesture_samples()    — graphiques 3D de trajectoires
  - standardize_gestures()    — centrage/réduction par geste
  - apply_pca()               — PCA sur les points standardisés
  - cluster_and_encode()      — k-means + encodage symbolique
  - knn_predict()             — logique de vote majoritaire k-NN
  - crossval_user_dependent() — leave-one-sample-out par utilisateur
  - save_fold_results()       — export CSV des accuracies par fold
  - Pipeline Domain 1         — sections 9.1 à 9.7 (identiques à 9.1 à 9.6 de Lenny)

  La LOGIQUE MATHÉMATIQUE de DTW et d'Edit Distance est inchangée :
  mêmes récurrences, même pondération diagonale ×2 pour DTW,
  même matrice de Levenshtein pour Edit Distance.


--------------------------------------------------------------------------------
RÉSUMÉ DES APPORTS
--------------------------------------------------------------------------------

  Problème identifié            Solution apportée            Impact attendu
  ──────────────────────────────────────────────────────────────────────────
  DTW ~2h+ de calcul            @njit + Parallel             ~10-50x plus rapide
  Edit Distance ~2h de calcul   @njit + Parallel             ~10-50x plus rapide
  Domain 4 absent               Sections 10.1-10.5           Pipeline complet
  Wilcoxon absent               wilcoxon_test()              Conformité consignes
  ──────────────────────────────────────────────────────────────────────────

================================================================================
```